DOCENTE	**Fabio ABBA’**
UNITA’ FORMATIVA	**Competenze informatiche/digitali**
ARGOMENTO	**RESTful API**

Sicurezza nelle applicazioni web



Docente: **Fabio ABBA'**

L'importanza della sicurezza web

L'argomento di oggi è della massima importanza nello sviluppo di applicazioni web moderne.

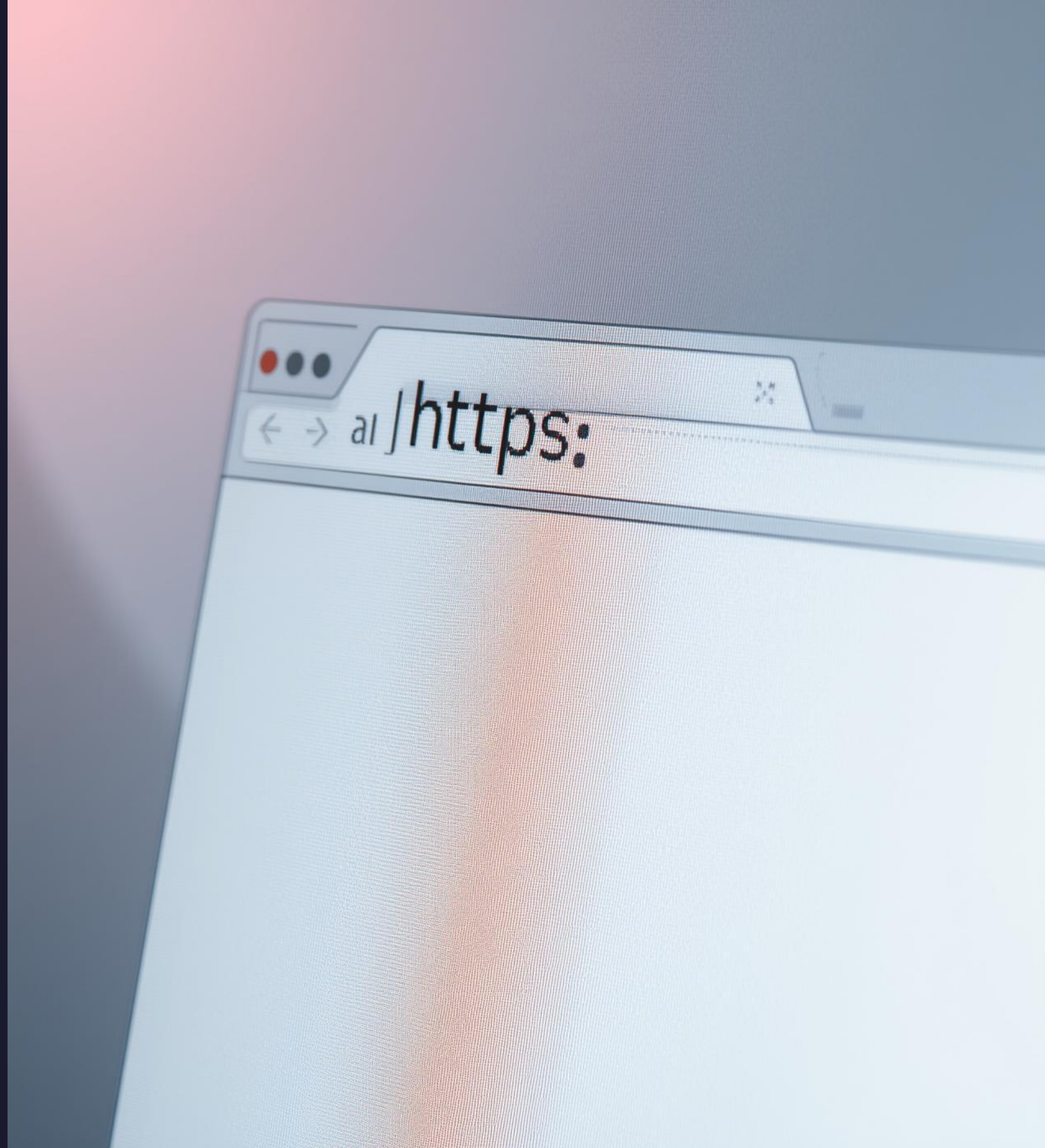
Immaginate di costruire una casa con muri solidi e una porta blindata, ma di lasciare la chiave sotto lo zerbino: questa è un'applicazione che non applica standard di sicurezza, cioè è facilmente violabile. Le vulnerabilità possono portare a furti di dati, danni reputazionali e perdite finanziarie.



HTTPS: Il lucchetto del web

Immaginiamo di essere in un bar affollato. Se parlo con il mio amico a voce alta, tutti possono sentire la nostra conversazione. Questo è HTTP: i dati viaggiano in chiaro sulla rete, chiunque può intercettarli e leggerli.

Ora immaginiamo di parlare in una lingua segreta che solo io e il mio amico conosciamo. Anche se qualcuno ascolta, non capisce nulla.
Questo è **HTTPS**: i dati sono crittografati.



Sicurezza Web

HTTPS e i suoi vantaggi

HTTPS è la versione sicura del protocollo HTTP. Aggiunge un livello di crittografia chiamato TLS (Transport Layer Security), il successore di SSL.

Crittografia

La **crittografia** assicura che i dati scambiati tra client e server siano cifrati. Se un malintenzionato intercettasse la comunicazione (attacco "Man-in-the-Middle"), vedrebbe solo dati incomprensibili.

Integrità

L'integrità garantisce che i dati non vengano alterati durante il trasferimento. Se un attaccante tentasse di modificare i dati, il sistema rileverebbe l'anomalia e respingerebbe la comunicazione.

Autenticazione

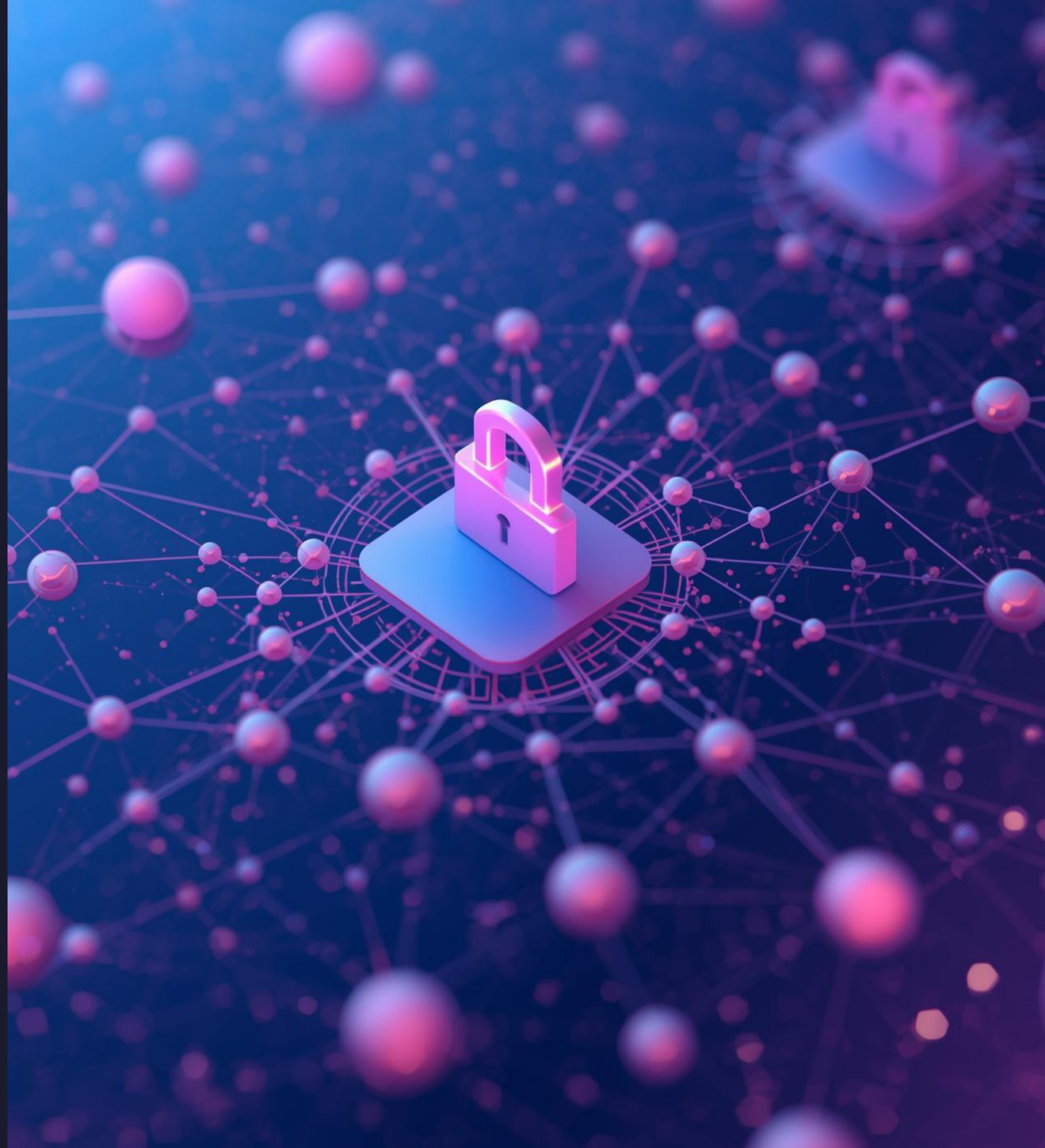
L'autenticazione verifica che le comunicazioni avvengano con il server corretto. Utilizzando certificati digitali, si assicura che gli utenti si connettano solo a fonti affidabili.



Autenticazione e autorizzazione

Un altro pilastro della sicurezza delle applicazioni web è la combinazione di **autenticazione** (chi sei?) e **autorizzazione** (cosa puoi fare?).

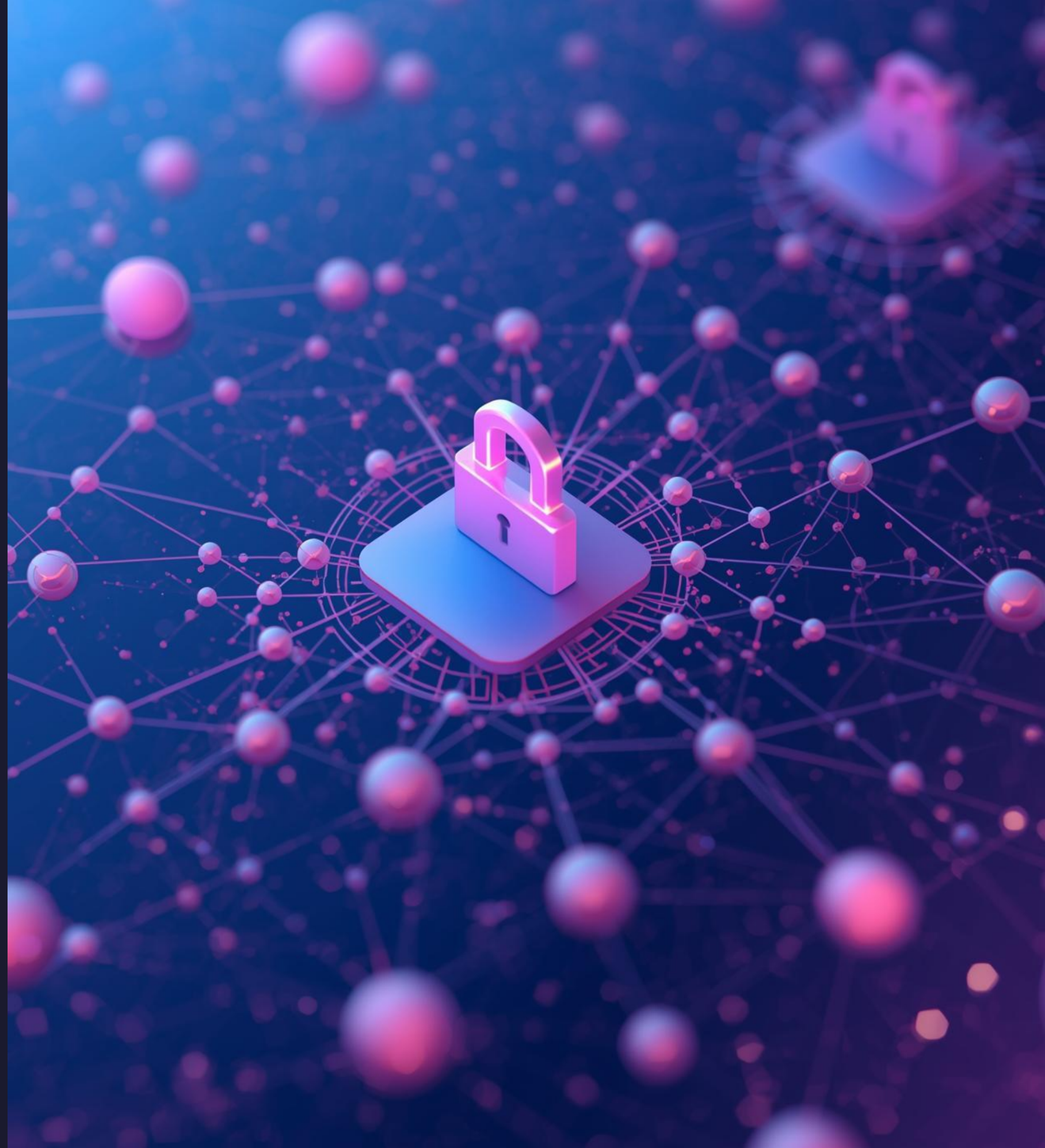
La strategia più comune e adatta alle API REST stateless è quella basata sui JSON Web Tokens (JWT).



JSON Web Token

Un JSON WebToken è una stringa che contiene (codificate) le informazioni che riguardano l'utente (idenificativo, ruolo/permessi), specifiche del token (emissione e scadenza), ed altre info customizzabili a seconda del contesto applicativo.

Tali informazioni sono firmate lato server secondo la specifica JSON Web Signature(JWS), quindi il server è in grado di riconoscere se sono state generate da lui o meno.



Autenticazione JWT

Fondamenti del processo di autenticazione

Login

L'utente invia le sue credenziali (es. email e password) a un endpoint come “/auth/login”.

Verifica

Il server verifica le credenziali; se sono valide, il server crea un JWT, ossia una stringa codificata che contiene informazioni sull'utente (es. ID, ruolo) e una firma digitale.

Invio token

Il server invia il token JWT al client, permettendo così all'utente di accedere a risorse protette.



Autenticazione JWT

Gestione delle richieste sicure

Richieste Autenticate

Il client deve includere il token JWT nell'header Authorization per tutte le richieste a endpoint protetti, garantendo che solo gli utenti autenticati possano accedere alle risorse.

Authentication: Bearer <il_tuo_jwt>

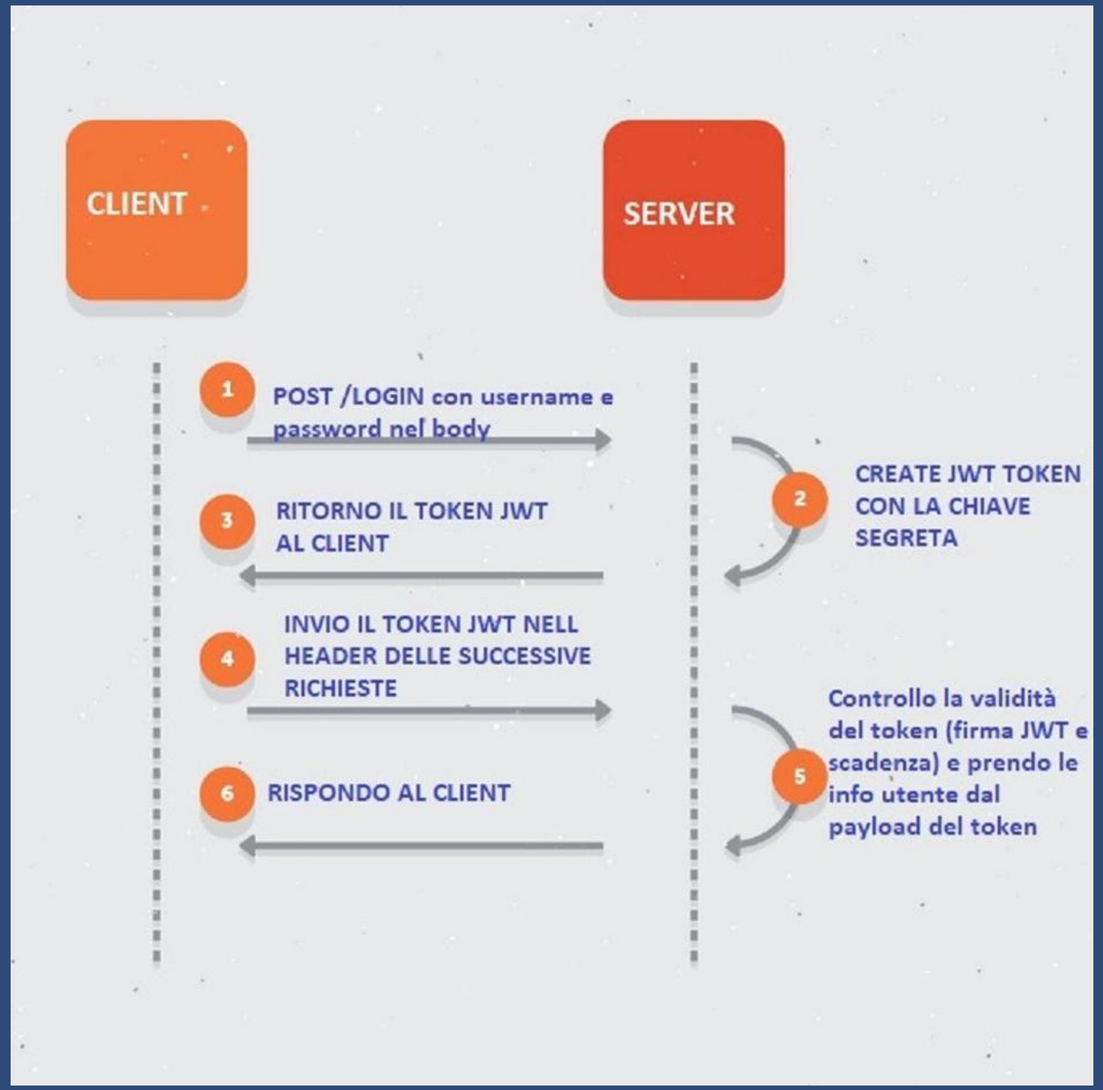
Verifica Token

Prima di processare una richiesta, il server verifica la firma del token. Se valida, estrae i dettagli utente; altrimenti, respinge la richiesta con un errore 401 Unauthorized.

Risposta al Client

Una volta verificato il token, il server invia al client le informazioni richieste, completando il ciclo di autenticazione.

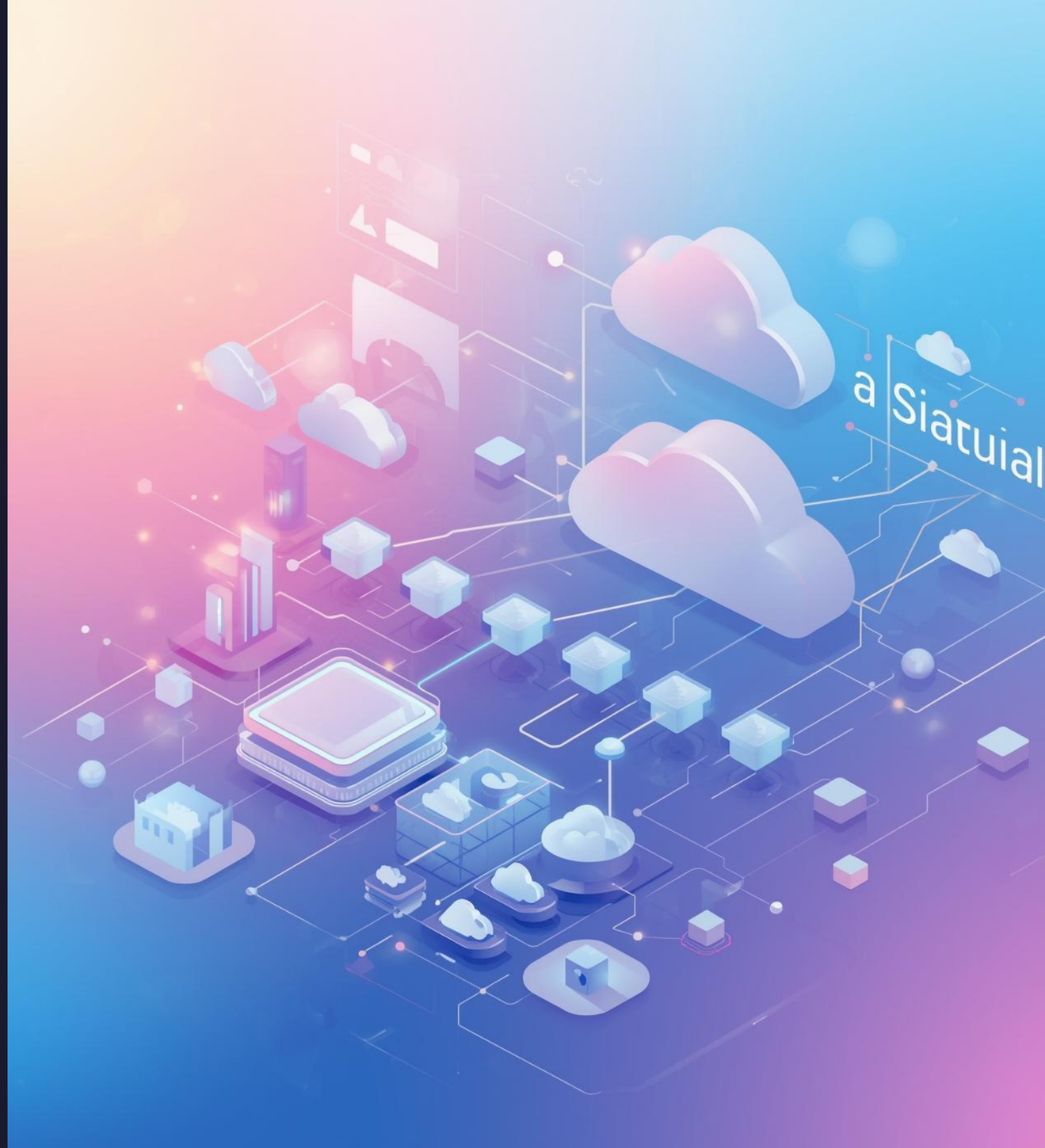



CLIENT
SERVER
1
POST /LOGIN con username e password nel body
2
CREATE JWT TOKEN CON LA CHIAVE SEGRETA
3
RITORNO IL TOKEN JWT AL CLIENT
4
INVIO IL TOKEN JWT NELL HEADER DELLE SUCCESSIVE RICHIESTE
5
Controllo la validità del token (firma JWT e scadenza) e prendo le info utente dal payload del token
6
RISPONDO AL CLIENT

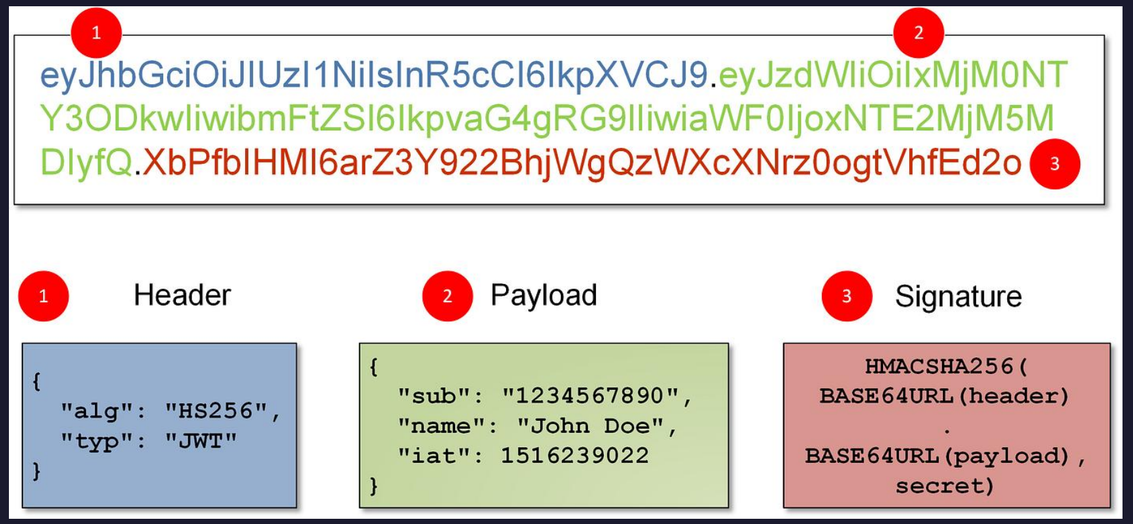
Vantaggi dell'uso di JWT

Stateless: Il server non ha bisogno di memorizzare informazioni sulla sessione. Ogni token è auto-contenuto. Questo rende l'architettura scalabile.

Portabilità: Un token può essere usato per accedere a diverse API o microservizi.



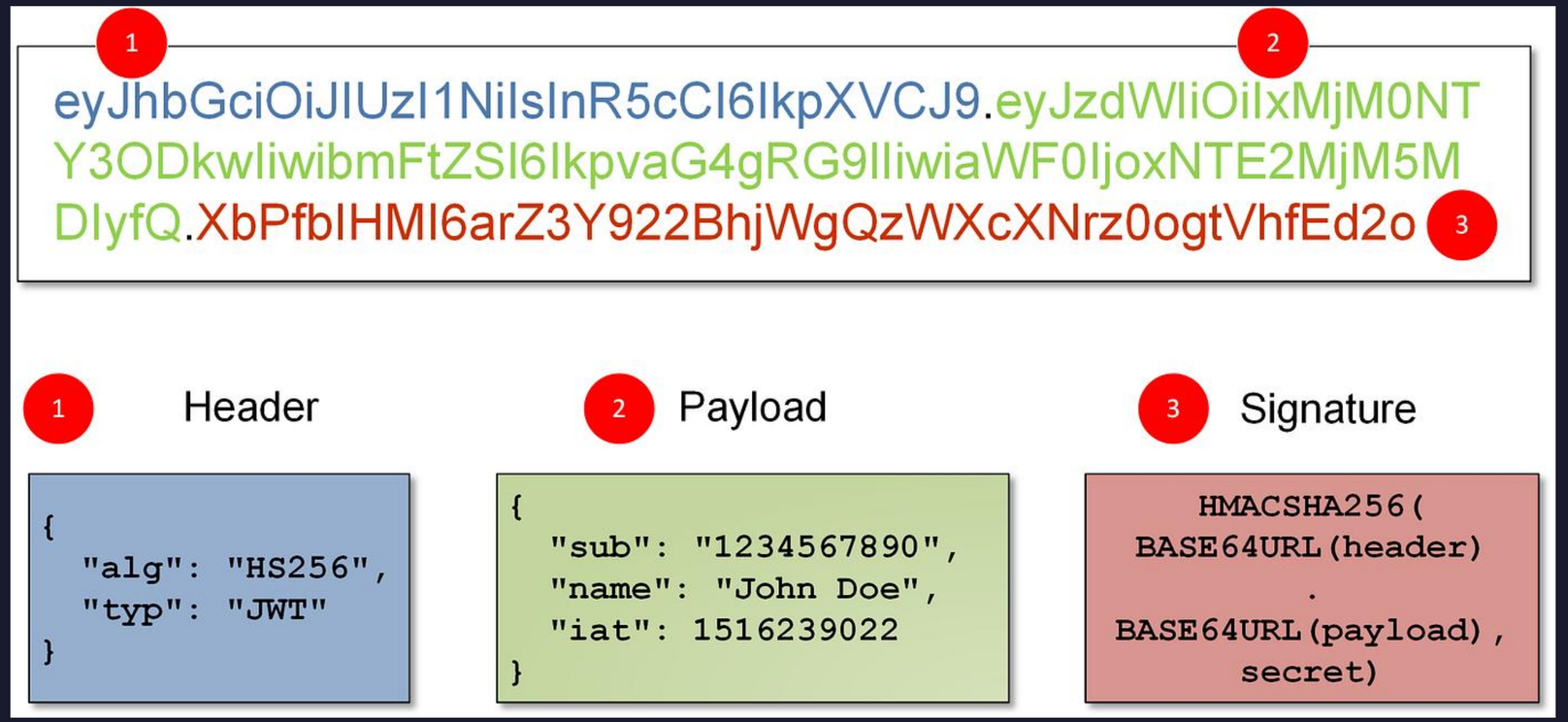
Analisi del JWT



Come si evince dall'esempio, identifichiamo 3 parti in un token JWT, separate da un punto:
Header, **Payload** e **Signature**.

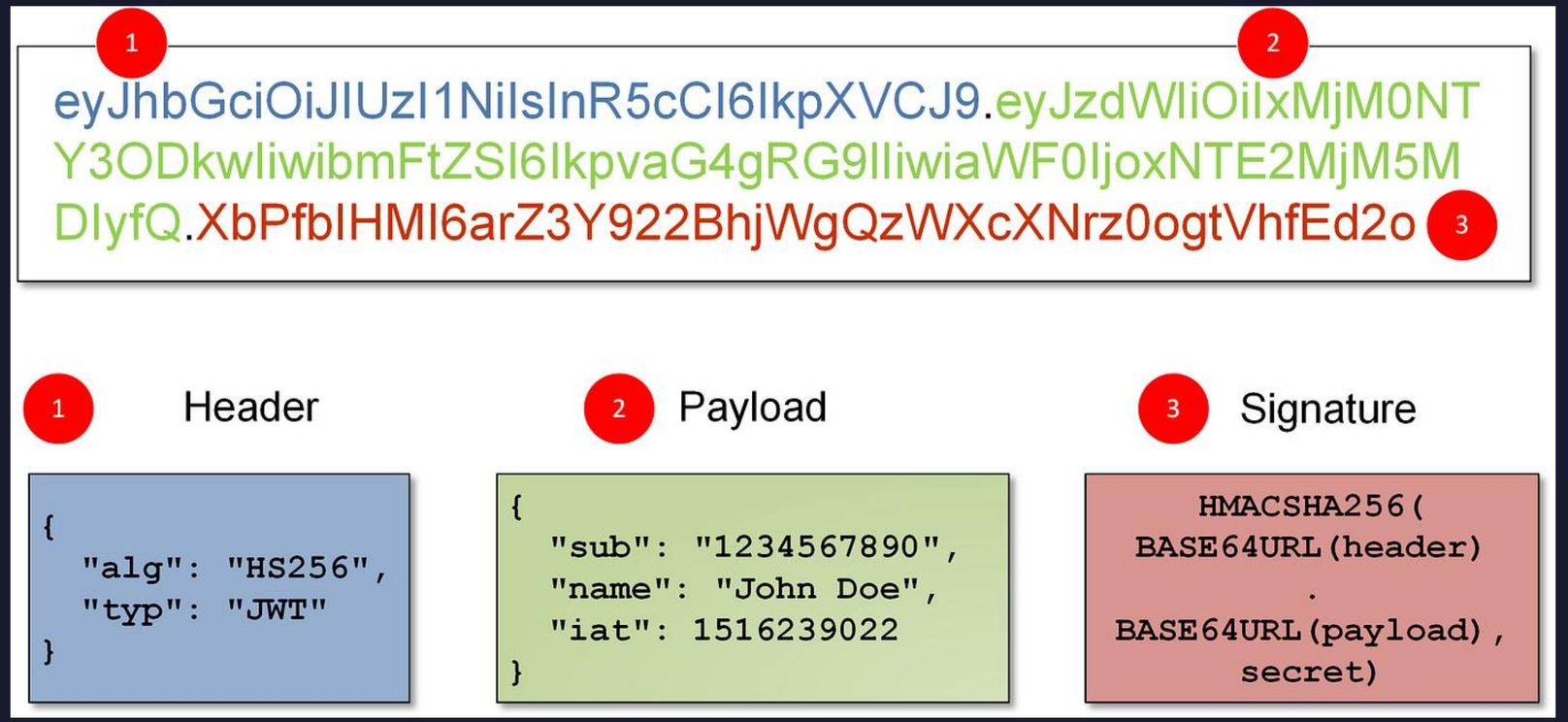


Analisi del JWT: Header



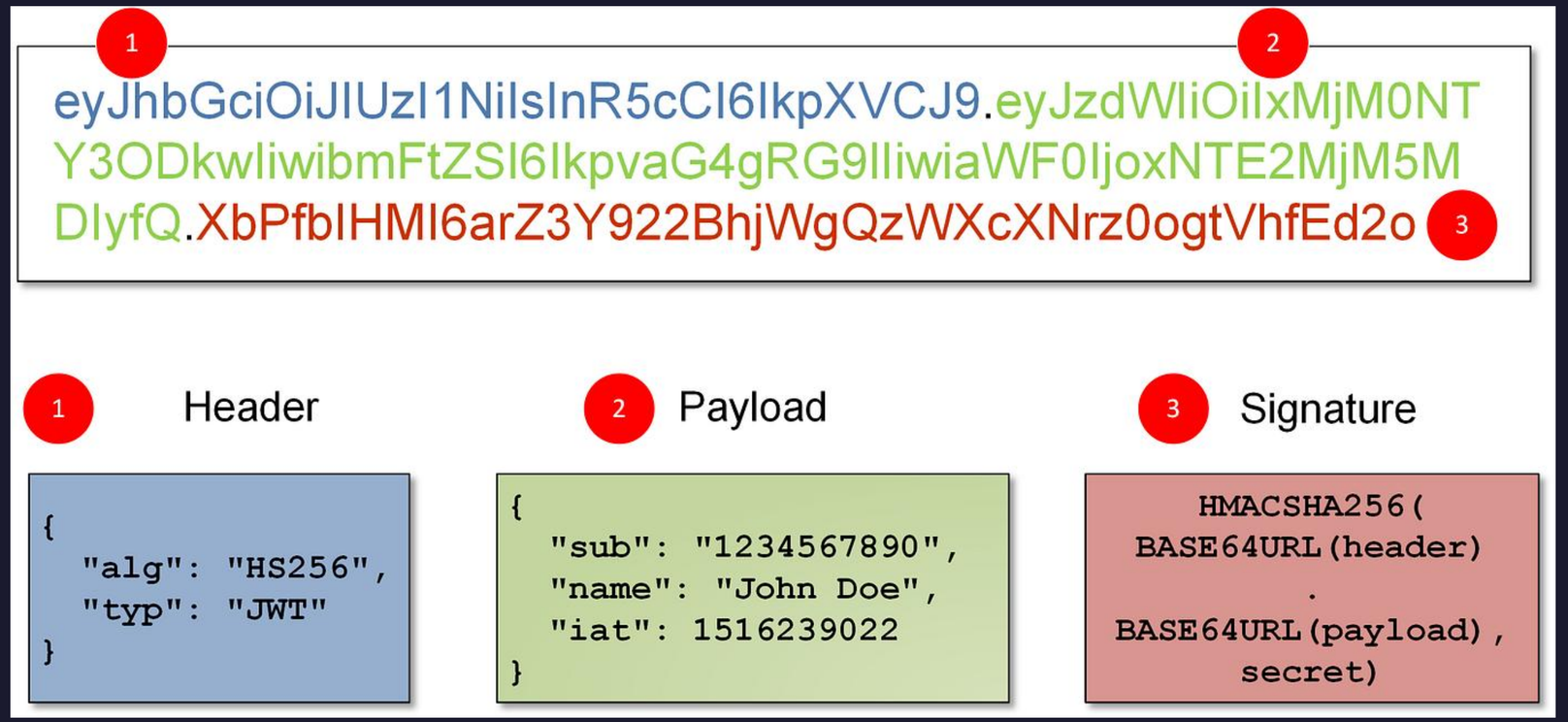
Decodificando la prima parte del token, scopriamo che si riferisce ad un json che contiene le due voci “typ” e “alg”. Il primo ha come valore sempre “JWT”, mentre il nodo “alg” contiene il nome dell’algoritmo usato per il token. Nell'esempio è stato usato l’algoritmo “HMAC-SHA256”.

Analisi del JWT: Payload



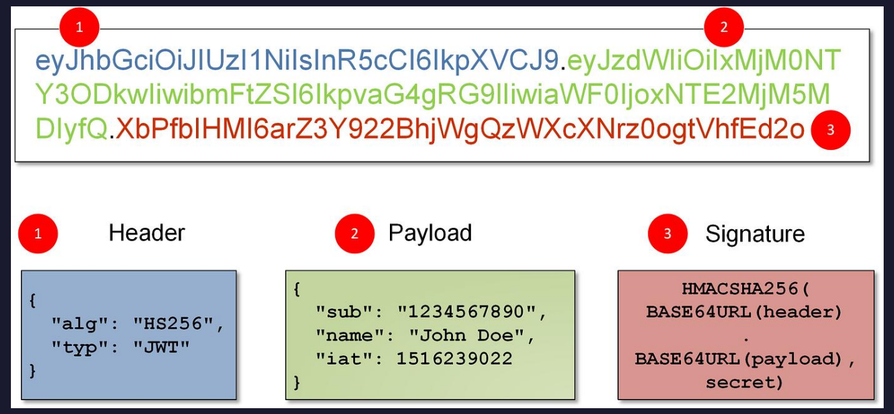
La seconda parte del token è il corpo vero e proprio, che contiene dei dati variabili in base al contesto, in formato json e codificati in base64. Nel nostro esempio il payload contiene nome e id dell'utente e il timestamp dell'emissione del token, ma manca la scadenza del token (!), il che costituisce una potenziale vulnerabilità.

Analisi del JWT: Signature



La terza parte del token è la firma, che assicura l'integrità del token; non è altro che il risultato di una funzione hash 256 che prende in input la codifica base64 dell'header concatenandola con un punto alla codifica base64 del payload, il tutto codificato con una "chiave segreta" definita nell'applicazione, e che solo il server conosce.

Attenzione!



Nota importane: la firma è l'unica parte crittografata del token; le informazioni contenute del payload sono decifrabili facilmente da chiunque entri in possesso del token: **MAI** inserire informazioni sensibili (password, informazioni personali, ...) nel payload!



Autorizzazione nelle applicazioni web

L'autorizzazione definisce i permessi degli utenti: cosa possono fare e a quali risorse possono accedere. È fondamentale per garantire che solo gli utenti autorizzati abbiano accesso a funzioni sensibili e dati protetti.

Ad esempio, un utente normale può leggere i prodotti, ma solo un admin può crearli o eliminarli.

L'autorizzazione solitamente si implementa con un middleware che viene eseguito dopo l'autenticazione e che si applica alle rotte definite nell'applicazione.



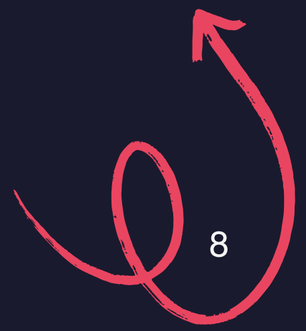
Best practices

- Usare sempre HTTPS in produzione
- Non salvare mai password in chiaro
- Usare JWT per l’autenticazione
- Impostare una scadenza ed eventualmente una rotazione dei token
- Mantenere le informazioni sensibili (JWT_SECRET, API keys) fuori dal codice in un file .env
- Dare agli utenti solo i permessi strettamente necessari (autorizzazione)



Risorse utili

- How To Use JSON Web Tokens (JWTs) in Express.js | DigitalOcean
- Mastering JWT authentication in Express - GeeksforGeeks





Grazie per l'attenzione!

*Docente: **Fabio Abbà***

fabio.abba@tc-web.it

Credits: Franco Grivet

franco.grivet@tc-web.it

Intervento realizzato da

ITS TECNOLOGIE DELL'INFORMAZIONE E DELLA COMUNICAZIONE

TC-Web Technology Consulting

Via Giovanni Schiaparelli 14,
10148 – Torino (TO)
E-mail: info@tc-web.it
Tel: 011 0466685